

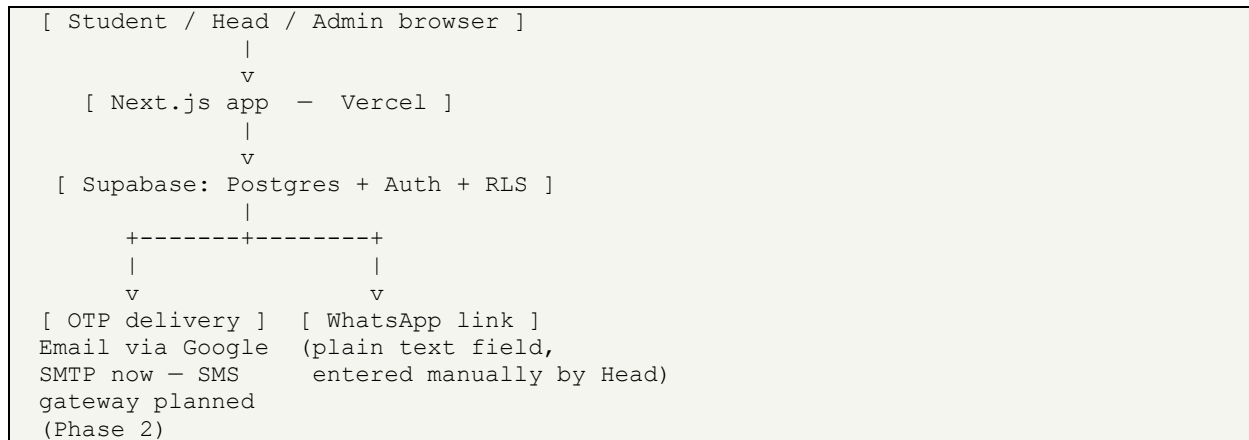
University Denomination Directory

System Design Document — v2 (working title; final product name pending)

Companion to the Requirements Document. Covers architecture, tech stack, database schema, and implementation decisions for the technical build.

1. Architecture Overview

A three-tier web architecture: a Next.js frontend, a Supabase (Postgres) backend handling data and auth, and an OTP delivery channel currently implemented as email via Google SMTP, with a planned upgrade to an SMS gateway once volume outgrows email's practical limits. There is no integration with WhatsApp's API in any form — WhatsApp group links are created manually by Chapter Heads and stored as plain text.



2. Tech Stack

Layer	Choice	Why
Frontend	Next.js (React)	Matches team's existing JavaScript/React comfort
Hosting (frontend)	Vercel	Free tier, deploys directly from Next.js, no server management
Backend & database	Supabase (Postgres)	Relational model fits the data (foreign keys throughout); built-in Row-Level Security maps directly onto chapter-scoped permissions; free tier suits a pilot
Auth	Supabase Auth (extended for email OTP) for students; email + password for Heads/Admin	Keeps auth in the same platform as the database; RLS policies can read the logged-in user's role directly; email now doubles as the student's unique ID
OTP delivery (v1)	Email via Google SMTP	Free, no gateway account needed to get started; fits the small/undetermined budget. Ceiling: roughly 500 emails/day on a standard Gmail account
OTP delivery (planned upgrade)	SMS via Hubtel or Arkesel	Africa/Ghana-focused gateways — typically better local delivery and lower per-message cost than international providers like Twilio. Adopt once email volume/reliability becomes a constraint
WhatsApp integration	None — plain text field	Avoids WhatsApp Business API entirely, as decided; the Head creates the group manually and pastes the invite link in

3. Database Schema

Postgres, hosted on Supabase. All tables use RLS to enforce the chapter-scoped permission model at the database layer — not just in application code.

3.1 denominations

```
create table denominations (  
  id          uuid primary key default gen_random_uuid(),  
  name        text not null,  
  description  text,  
  created_at  timestampz default now()  
);
```

3.2 chapters

```
create type chapter_status as enum ('draft', 'coming_soon', 'active');  
  
create table chapters (  
  id                uuid primary key default gen_random_uuid(),  
  denomination_id   uuid references denominations(id) not null,  
  campus            text not null check (campus in ('Main Campus', 'Essikado')),  
  status            chapter_status not null default 'draft',  
  meeting_day       text,  
  meeting_time      text,  
  location          text,  
  description       text,  
  whatsapp_link     text,  
  created_at        timestampz default now()  
);
```

3.3 heads

```
create table heads (  
  id                uuid primary key default gen_random_uuid(),  
  chapter_id        uuid references chapters(id) unique not null,  
  name              text not null,  
  email             text unique not null,  
  phone             text,  
  created_at        timestampz default now()  
  -- password handled by Supabase Auth, not stored here directly  
);
```

3.4 students

```
create table students (  
  email            text primary key, -- login identifier  
  phone_number     text,             -- contact field only  
  name             text not null,  
  program          text,  
  level            text,  
  hall             text,  
  current_chapter_id uuid references chapters(id),  
  created_at       timestampz default now()  
);  
  
-- Enforces single-chapter exclusivity: a student can only have  
-- one non-null current_chapter_id at a time. Registration logic  
-- checks this before insert/update, since it depends on the  
-- "student must be removed by their current head first" rule  
-- rather than a pure uniqueness constraint.  
-- Exclusivity is now keyed on email (the login identifier),  
-- not phone_number.
```

3.5 system_admins

```
create table system_admins (  
  id          uuid primary key default gen_random_uuid(),  
  name        text not null,  
  email       text unique not null,  
  created_at  timestamptz default now()  
  -- password handled by Supabase Auth  
);
```

4. Row-Level Security (permission enforcement)

RLS policies (Postgres) enforce the access rules directly on the database, so even a bug in the frontend can't leak another chapter's data:

- Students: can select/update only the row in students matching their own authenticated email address.
- Heads: can select/update only rows in students where `current_chapter_id` = their own `chapter_id` (looked up via their authenticated head record); can update only their own row in chapters.
- System Admin: can insert/update denominations and chapters, and insert rows in heads; does not get a blanket policy granting read access to students — matching the "no default roster access" decision.

5. Authentication & Session Flow

5.1 Students (passwordless)

- Student enters their email address → backend triggers an OTP email via Google SMTP (v1). The OTP delivery step is intended to be built as a swappable interface, so moving to an SMS gateway later doesn't require touching the verification logic.
- Student enters the OTP → backend verifies, issues a long-lived session token (e.g. 60–90 days).
- Session token stored client-side (secure, httpOnly cookie preferred over local storage) and silently renewed on activity.
- Re-authentication (fresh OTP) only required on new device or after expiry.

5.2 Chapter Heads & System Admin (password-based)

- Standard email + password login via Supabase Auth.
- Minimal password-reset flow required: email-based reset link (not yet built in v1 scope, but needed before launch since a locked-out Head has no self-service path otherwise).

6. Cost Considerations

- Vercel (frontend hosting): free tier sufficient for pilot traffic.
- Supabase (database + auth): free tier sufficient for pilot scale; costs only appear at meaningfully larger usage.
- Email OTP (v1): effectively free via Google SMTP, within its sending limit (~500/day on a standard account) — the main reason this was chosen as the starting point given budget isn't settled yet.
- SMS OTP (planned upgrade): the only per-transaction cost anticipated in this design once adopted — scales with student sign-ins, billed per message by Hubtel/Arkesel. Worth getting an actual quote from both before committing, since Ghana-specific delivery rates and pricing can differ.
- No WhatsApp API costs, since no WhatsApp integration exists in this design.

7. Open Items — Carried Over & New

- Final choice between Hubtel and Arkesel for the future SMS upgrade — needs a real quote/reliability comparison when that phase starts.

- Exact session expiry length and whether it's force-invalidated when a student leaves a chapter.
- Password recovery flow for Heads/Admin — design only, not yet built.
- Rate-limiting OTP requests (to prevent abuse from repeated requests to the same or made-up addresses — relevant to both the Google SMTP sending cap now and SMS cost later) — not yet designed.
- Concrete trigger for the email→SMS upgrade (e.g. a specific daily-volume threshold vs. a fixed calendar point) — not yet decided.
- Whether membership history is tracked in a separate table, or only the current chapter (current schema only tracks current).
- Whether "coming soon" chapters capture student interest ahead of going active (from the Requirements Document's open items).